FH University of Applied Sciences
TECHNIKUM WIEN

ASSIGNMENT

In Medical Engineering & eHealth

Assignment in Sequence Alignment

By: Sebastian Schedl

Student Number: me25m019

Supervisor: Dr Prabath Jayathissa

Wien, June 16, 2026

Contents

Abstract

This report shows the implementation of four sequence alignment methods in Python. The code is well-commented and easy to read. The algorithms are: Needleman-Wunsch (Global Alignment), Smith-Waterman (Local Alignment), Gotoh (Global with Affine Gaps), and a simple BLAST-style Seed-and-Extend heuristic. We test these methods with three datasets: short related proteins, distant homologs, and pieces of the *E. coli* genome. We also measure the computational time and memory used by each algorithm, and we check how different parameters change the score. The results show that exact dynamic programming methods use $O(n^2)$ time and memory, but the heuristic method is much faster and uses very little memory.

1 Introduction and Background

Sequence alignment is very important in bioinformatics. Finding similarities between DNA, RNA, or proteins helps us understand biology better, like finding closely related genes or building family trees. We usually divide methods into global alignment (matching the whole sequence) and local alignment (finding small matching parts) [1], [2].

In practice, we must choose between getting the best accuracy and saving computer time. Exact dynamic programming (DP) algorithms always find the best score based on the rules. But they take too much memory and time for very long sequences like whole genomes. Because of this, it is common to use faster heuristic methods [3].

The goal of this assignment is to write simple Python code for four basic alignment methods. We will test how much time and memory they use, and discuss when to use which method.

2 Algorithms and Theory

2.1 Exact Dynamic Programming Methods

2.1.1 Needleman-Wunsch (Global)

The Needleman-Wunsch algorithm finds the best global alignment. It aligns two sequences from the start to the end [1]. It builds a matrix using match, mismatch, and gap scores. After the matrix is filled, a traceback finds the best matched sequence.

2.1.2 Smith-Waterman (Local)

The Smith-Waterman algorithm is used for local alignment [2]. It is very similar to Needleman-Wunsch, but the matrix scores cannot go below zero. If a score becomes negative, it is set to zero. The traceback starts at the highest number in the whole matrix and stops when it reaches a zero. This isolates the best matching part between two sequences.

2.1.3 Gotoh (Affine Gap Penalties)

Normal algorithms give the same penalty for one 10-character gap as for ten separate 1-character gaps. In biology, one big gap is more common than many small gaps. Gotoh fixes this by using different penalties for opening a gap and extending a gap [4]. Our code does this by using three separate matrices (one for match, one for gap in sequence 1, and one for gap in sequence 2).

2.2 Heuristic/Approximate Methods

2.2.1 BLAST-style Seed-and-Extend

Exact DP algorithms are too slow ($O(n^2)$). To make it run much faster, we can use a heuristic method like BLAST [3]. It splits the sequence into small exact words called k-mers (seeds). When it finds the same seed in both sequences, it tries to make the match longer to the left and right without adding any gaps.

3 Implementation Details

The code is written in a simple object-oriented way. We use classes and dynamic file paths so it runs on any computer without problems. The files in the `Implementation/` folder are:

- `Config.py`: Stores static parameters like scores and gap penalties.
- `SequenceReader.py`: Reads `.fasta` dataset files.
- `NeedlemanWunsch.py`: Global Alignment DP algorithm.
- `SmithWaterman.py`: Local Alignment DP algorithm.
- `Gotoh.py`: Gotoh algorithm with 3 matrices.
- `BasicSeedAndExtend.py`: Heuristic seed matcher.
- `ExperimentRunner.py`: Runs the experiments to measure time and memory.
- `Main.py`: The script to start the program.

4 Experimental Design

We measure the running time in milliseconds and the peak memory in KB. We use the Python `time` and `tracemalloc` libraries for this.

4.1 Standard Configurations

Our standard scoring rules are:

- Match score: $+2$
- Mismatch score: -1
- Gap penalty: -2

For Gotoh's affine gaps, the Gap Open penalty is -3, and Gap Extension is -1.

4.2 Datasets

1. **Short Related Proteins:** Tests algorithms on normal, similar sequences.
2. **Distant Homologs:** Tests local alignment on sequences that are not very similar and have many mutations.
3. **E. coli Genomic Slices**: We take 200bp pieces from a long genome to see how fast the algorithms are.

5 Results

5.1 Performance Benchmarks

Table 1: Dataset 1: Short related proteins.fasta (∼150bp segments)

Algorithm	Score	Runtime (ms)	Peak Memory (KB)
Needleman–Wunsch (Global)	28	22.37	739.06
Smith–Waterman (Local)	34	3.13	170.26
Gotoh (Affine Gap)	27	58.68	2195.45
Basic Seed-and-Extend (Heuristic)	5	0.19	17.13

Table 2: Dataset 2: Distant homologs.fasta (∼200bp segments)

Algorithm	Score	Runtime (ms)	Peak Memory (KB)
Needleman–Wunsch (Global)	-107	44.09	1630.50
Smith–Waterman (Local)	9	6.06	372.99
Gotoh (Affine Gap)	-105	135.50	4752.83
Basic Seed-and-Extend (Heuristic)	4	0.28	27.75

Table 3: Dataset 3: E. coli K-12 Genome Slices (200bp exact)

Algorithm	Score	Runtime (ms)	Peak Memory (KB)
Needleman–Wunsch (Global)	67	36.18	1241.73
Smith–Waterman (Local)	82	6.30	373.51
Gotoh (Affine Gap)	59	92.76	3311.83
Basic Seed-and-Extend (Heuristic)	7	0.49	16.21

5.2 Parameter Sensitivity

We tested Needleman-Wunsch with three different scoring rules on the short proteins dataset. The standard score was 28.

- **Set 1 (Match: 2, Mis: -1, Gap: -5):** Strong gap penalty. The score drops to -31. The algorithm cannot use gaps easily, so the alignment is bad.
- **Set 2 (Match: 2, Mis: -1, Gap: 0):** Free gaps. The score goes very high to 98. The algorithm just uses too many gaps to match single letters.
- **Set 3 (Match: 1, Mis: -5, Gap: -1):** Strong mismatch penalty. The score goes down to -53. The algorithm prefers to use gaps instead of taking a mismatch.

6 Discussion and Conclusions

The test results show what we expect from the theory. Needleman-Wunsch is an okay global method, but it gets very low scores on distant homologs. This happens because mutations give negative points over the whole sequence. Smith-Waterman is much better here. It gets positive scores because it only aligns the good parts and ignores the bad parts.

The memory and time results show the problems of DP algorithms. Needleman-Wunsch needs about 1 MB of memory just for 200bp sequences. Gotoh needs exactly three times more memory (3.31 MB) because it uses three matrices (M, X, Y). On the other hand, the Seed-and-Extend heuristic is extremely fast (less than 1 millisecond) and uses less than 20 KB of memory. This is exactly why heuristic methods like BLAST are used for big databases today.

The parameter tests show that scores are very important to get a biological meaning. If we change the gap penalties, the algorithm gives a completely different output. We must choose the parameters very carefully depending on the biology problem.

Bibliography

[1] S. B. Needleman and C. D. Wunsch, “A general method applicable to the search for similarities in the amino acid sequence of two proteins,” *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970. DOI: 10.1016/0022-2836(70)90057-4.

[2] T. F. Smith and M. S. Waterman, “Identification of common molecular subsequences,” *Journal of Molecular Biology*, vol. 147, no. 1, pp. 195–197, 1981. DOI: 10.1016/0022-2836(81)90087-5.

[3] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990. DOI: 10.1016/S0022-2836(05)80360-2.

[4] O. Gotoh, “An improved algorithm for matching biological sequences,” *Journal of Molecular Biology*, vol. 162, no. 3, pp. 705–708, 1982. DOI: 10.1016/0022-2836(82)90398-9.

List of Tables

A Code Excerpts

Implementation files overview:

- `NeedlemanWunsch.py`: Global exact DP matrix structure.
- `SmithWaterman.py`: Local exact DP with non-negative resets.
- `Gotoh.py`: Mathematical framework for affine gap matrices.
- `BasicSeedAndExtend.py`: k-mer lookup dictionary and linear extensions.
- `ExperimentRunner.py`: Tracemalloc logging and benchmarking wrappers.

Repository URL: https://github.com/SebastianFH-png/IBSY-Assignment4.git